

THE UNIVERSITY OF HONG KONG

DEPARTMENT OF COMPUTER SCIENCE 2025-26

COMP4801

Final Year Project

Interim Report

BAE Junyoung (3035716464)

Supervisor: Dr. Chim Tat Wing

Topic: Automated Job Application Tracker

1. Project Background

The job market has become increasingly competitive, with students and professionals often applying to dozens or even hundreds of positions across different companies and platforms. According to career industry statistics, the average job seeker submits over 100 applications during their search [1]. Tracking the progress of each application—from initial submission through various assessment stages to final outcome—becomes extremely difficult when information is scattered across email inboxes. This fragmentation leads to missed opportunities, forgotten follow-ups, and an inability to learn from patterns in one's job search.

The typical job application process begins with candidates searching for opportunities on platforms such as LinkedIn, Glassdoor, and Indeed. These platforms help users discover jobs but do not manage the full application lifecycle. Once a suitable position is found, applicants are redirected to the company's career portal to submit their application, often through Applicant Tracking Systems (ATS) such as Workday, Greenhouse, or Lever. After submission, each company follows its own recruitment process, which often differs significantly. Some may begin with resume screening followed by an aptitude test, while others may move directly to a coding challenge or interview. This results in multiple parallel processes with varying entry points and formats, from single interviews to multi-round assessments across video, on-site, and technical formats.

Existing job portals provide limited tracking features, but only for applications submitted through their platforms. They do not cover applications made via independent company portals or email, leaving a significant gap once candidates move beyond the job discovery stage. Common problems faced by job seekers include application confirmations and interview invites getting buried among hundreds of other emails, spreadsheets becoming outdated quickly as users forget which companies they applied to or what stage they are at, and a general lack of visibility that prevents applicants from identifying patterns in their job search or understanding their application success rate.

This project addresses that gap by developing a web application that integrates with Gmail to automatically track job-related emails. By extracting and classifying messages into recruitment stages (Applied,

Screening, Test, Interview, Offer/Rejection), the system visualizes the application pipeline across companies. With the Gmail API and AI powered by GPT-4o-mini, the system retrieves and classifies emails, generates summary statistics such as conversion rates between stages, and visualizes application progress to highlight bottlenecks. In addition, a conversational chatbot assists users with common queries and provides tailored recommendations based on their actual application data.

2. Project Objectives

To address the challenges faced by job seekers, this final-year project ultimately aims to reduce the manual effort required to track job applications and provide actionable insights into the job search process. Leveraging the Gmail API and Azure OpenAI's GPT-4o-mini model, the project seeks to automate the extraction and classification of job-related emails, enabling users to focus on preparing for interviews rather than managing spreadsheets.

To achieve the primary objective, the system implements a personalized job application tracking dashboard that leverages Natural Language Processing (NLP) and a two-layer classification system to deliver accurate stage detection and company extraction. This application aims to enable job seekers to understand their application pipeline at a glance and make informed decisions about where to focus their efforts. The proposed solution accomplishes the following:

1. Connects to a user's Gmail account through OAuth 2.0 authentication and automatically retrieves job-related emails using Gmail's advanced query filters. The platform operates in read-only mode using the `gmail.readonly` scope to ensure user privacy and security, with no raw email content stored on any server.
2. Classifies emails into specific stages of the recruitment pipeline using a hybrid approach combining rule-based pattern matching for obvious cases (such as "thank you for applying" indicating application submission) and AI-powered analysis for ambiguous situations. The stages tracked include Application Submitted, Aptitude Test, Simulation Test, Coding Test, Video Interview, Human Interview, Offer, and Rejection.
3. Visualizes the user's job application progress through multiple interactive components including a Sankey diagram showing flow from application to final outcome, an application funnel displaying conversion rates between stages, a timeline view with company-grouped chronological history, and a statistics dashboard with key metrics and response rates.
4. Provides an AI-powered chatbot assistant that offers personalized advice based on the user's actual application data, capable of answering questions about application status, providing interview preparation tips for specific companies, analyzing patterns in the job search, and offering actionable recommendations.

The platform differentiates itself from existing solutions through several key features. The two-layer classification system ensures both speed and accuracy by using deterministic rules for clear-cut cases while reserving AI analysis for ambiguous emails. The system extracts real company names from ATS platform emails (for example, identifying "ION Group" from an email sent via hire.lever.co), handles multiple positions at the same company, and provides manual override capabilities for users to correct any misclassifications. Additionally, the permanent caching system with incremental date-range updates ensures that returning users experience near-instant load times while still receiving fresh data.

However, there are challenges posed by the complexity of implementing such a system. First, ATS platforms format emails differently, requiring continuous pattern updates to maintain classification accuracy. Second, when users apply to multiple positions at the same company, disambiguation becomes challenging as emails may not clearly indicate which position they reference. Third, personal recruiter emails sent from addresses like @gmail.com are harder to associate with companies. Finally, the current patterns primarily support English, limiting the system's utility for users applying to positions in non-English speaking regions.

3. Project Methodology

This section introduces the methodologies utilized in the project. It details the system architecture, the two-layer classification approach, the caching strategy, and various system components to achieve accurate email classification and real-time progress feedback.

3.1 System Architecture

The system follows a three-tier architecture separating concerns between frontend visualization, authentication handling, and email processing (see Figure 1). The frontend is built with React.js and TypeScript, providing an interactive dashboard with components for statistics display, Sankey diagram visualization, application timeline, and chatbot interaction. The styling uses Tailwind CSS for responsive design, while Recharts and Plotly.js handle the data visualizations.

The authentication layer runs on Render.com as a Flask backend, handling OAuth 2.0 authentication with Google's Gmail API. This server manages token storage and refresh, ensuring users remain authenticated across sessions. The processing layer runs locally as a separate Flask server, responsible for email classification using Azure OpenAI's GPT-4o-mini model, caching processed results, and serving the chatbot functionality. This separation ensures that sensitive email processing happens locally while authentication tokens are securely managed in the cloud.

Figure 1. *System Architecture Design showing the three-tier separation between Frontend (React), Render Backend (OAuth), and Local Server (AI Processing), with connections to Google APIs and Azure OpenAI.*

3.2 Two-Layer Classification System

The classification system employs a hybrid approach combining deterministic rules with AI-powered analysis, designed to maximize both speed and accuracy.

Layer A: Rule-Based Fast Filter. The first layer quickly eliminates non-job emails and pre-classifies obvious cases using pattern matching. Gmail queries pre-filter emails using terms like "application," "applying," and "interview" to reduce the volume of emails requiring processing. Known ATS domains such as workday.com, greenhouse.io, lever.co, and tal.net are identified to flag emails likely to be job-related. Stage detection patterns use regular expressions to identify clear indicators—for example, phrases like "thank you for applying" or "we've received your application" indicate application submission, while platform names like HackerRank or Codility indicate coding tests, and phrases like "unfortunately" or "regret to inform" indicate rejection.

Layer B: AI Refinement. The second layer handles ambiguous cases using GPT-4o-mini with carefully crafted prompts. This layer extracts real company names from ATS platform emails by analyzing the email subject and body to identify the actual employer rather than the ATS provider. For example, an email from "hire.lever.co" with subject "Thank you for your application to ION Group" would be correctly attributed to

ION Group. The AI layer also identifies job positions from email content, classifies stages for emails that don't match clear patterns, and handles the complexity of multiple positions at the same company by maintaining position-level tracking with indices.

3.3 Caching Strategy

The system implements a permanent cache with incremental date-range updates to optimize performance. Each user's processed data is stored locally in a JSON file containing the earliest and latest dates covered, the list of companies with their positions and stages, and aggregate statistics. When a user returns to the dashboard, the system checks whether the cached date range covers the requested period. If the cache fully covers the request, data is returned instantly in under one second. If the request extends to an earlier date, only the new date range is fetched and merged with existing data. A refresh button allows users to fetch emails since the latest cached date, ensuring they see recent updates without reprocessing their entire email history.

3.4 Real-Time Progress Feedback

The system uses Server-Sent Events (SSE) to provide real-time progress updates during email processing. When a user initiates email analysis, the frontend establishes an SSE connection to the local server, which then streams progress updates as processing proceeds. The stages reported include fetching email data with counts, scanning for job applications, detecting companies with a list of names found, AI analysis progress showing the current company being processed and overall progress (for example, "AI analyzing Morgan Stanley... 3/15"), stage classification, and final dashboard building. This approach keeps users informed during what could otherwise be a lengthy process, particularly for users with large email volumes or many job applications.

3.5 Security and Privacy

Privacy and security were primary considerations in the system design. The OAuth implementation uses the minimal `gmail.readonly` scope, ensuring the application cannot modify or delete emails. Authentication tokens are stored only on the Render backend server, never exposed to the frontend or stored locally. Email content is processed but not stored—only extracted metadata (company name, position, stage, dates) is cached. All processed data remains on the user's local machine, and HTTPS is enforced on all endpoints to protect data in transit.

4. Preliminary Results & Discussions

4.1 Progress Overview

The project has achieved significant progress, with all core features fully implemented and operational. The Gmail OAuth integration is complete and tested with multiple user accounts. Email retrieval and parsing successfully handles various email formats from different ATS platforms. Both the rule-based classification and AI classification layers are functioning, with the system correctly identifying companies and stages from real email data. The frontend dashboard displays all visualizations including the Sankey diagram, application funnel, timeline view, and statistics cards. The chatbot widget responds to user queries with context-aware answers based on their application data. Manual CRUD operations allow users to add, edit, and delete applications. The caching system provides instant load times for returning users, and SSE progress updates keep users informed during processing.

4.2 Frontend Implementation

The landing page introduces users to the application with a clear value proposition, explaining the problem of scattered job application emails and demonstrating how the system solves it. The hero section features a prominent Gmail connection button with trust indicators emphasizing read-only access and local data storage. A visual diagram illustrates the complex job application process, followed by sections describing the four key capabilities: automated tracking, interactive visualizations, performance analytics, and AI chatbot assistance. A three-step process explanation (Connect, Analyze, Track) helps users understand how to get started.

The dashboard provides a comprehensive view of the user's job applications (see Figure 2). The header displays connection status and provides a refresh button for fetching new emails. Statistics cards show key metrics including total applications, tests scheduled, interviews reached, pending applications, offers received, and rejections. The Sankey diagram visualizes the flow of applications from submission through various stages to final outcomes, making it immediately apparent where applications tend to drop off. The application funnel shows conversion rates between stages, while the timeline view groups events by company, showing the chronological progression of each application. Performance analytics display the overall response rate and other aggregate metrics.

Figure 2. *Dashboard Layout showing statistics cards, Sankey diagram, application funnel, timeline view, and performance analytics sections.*

When users click "Analyze Emails," the system displays a step-by-step progress indicator (see Figure 3). Each step shows its status (completed, in progress, or pending) along with relevant counts such as the number of emails found, applications detected, and companies identified. A progress bar provides visual feedback on overall completion, and the current operation is highlighted with details like which company is being analyzed.

Figure 3. *Processing Progress UI showing step-by-step status updates during email analysis.*

4.3 Backend Implementation

The backend successfully processes emails through a multi-stage pipeline. First, the Gmail API is queried with job-related search terms to retrieve candidate emails. Duplicate emails from reply chains are removed through deduplication. The Layer A filter applies rule-based classification to identify clear-cut cases and extract metadata. Layer B AI refines the classification using GPT-4o-mini for ambiguous emails and company name extraction. Results are organized by company and position through grouping, and finally stored in the cache for future retrieval.

Testing with real email data from various companies yielded encouraging results. Company extraction achieved 86% accuracy across 50 test emails, with most errors occurring on emails from lesser-known ATS platforms or direct recruiter communications. Application stage detection reached 82% accuracy, with the system particularly strong at identifying rejections (95% accuracy) and offers (100% accuracy on the limited sample of 5 offer emails). ATS email handling achieved 80% accuracy, with challenges primarily arising from non-standard email formats.

Several challenges were identified during testing. ATS platforms vary significantly in their email formatting, requiring continuous pattern updates as new formats are encountered. When users apply to multiple positions at the same company, the system sometimes struggles to correctly attribute follow-up emails to

the right position. Personal recruiter emails sent from generic email addresses like @gmail.com are difficult to associate with companies without additional context. Finally, the current pattern library primarily supports English, limiting accuracy for emails in other languages.

4.4 Chatbot Implementation

The AI chatbot provides personalized assistance based on the user's actual application data. When a user asks "How is my job search going?", the chatbot analyzes their application data and responds with a summary including the number of positions applied to across how many companies, how many applications have progressed to interviews, the overall response rate compared to typical benchmarks, and which companies have shown interest. The chatbot can also provide interview preparation tips for specific companies, identify patterns such as which types of roles have higher success rates, and offer actionable recommendations for improving the job search strategy.

The implementation passes the transformed application data from the frontend to the chatbot endpoint, allowing the AI to provide contextually relevant responses specific to each user's situation rather than generic advice.

4.5 Challenges and Solutions

Throughout development, several challenges were encountered and addressed. ATS email variations required building an expandable pattern library with AI fallback for unrecognized formats. Company name extraction was improved by training GPT-4o-mini with specific prompts that exclude ATS domains and focus on extracting the actual employer name from email subjects and bodies, achieving 86% accuracy. Processing latency was addressed through SSE streaming for real-time feedback and an aggressive caching layer that reduces repeat load times to under one second. Multiple positions per company are now handled through position-level tracking with indices, allowing the system to maintain separate records for each role. Session management across browser tabs was solved through cross-tab event broadcasting, ensuring that signing out in one tab updates all other open tabs.

5. Upcoming Schedules

After completing the core implementation phase, the project now moves into feature expansion and deployment. The immediate focus is on leveraging the application data collected from emails to provide actionable career insights through AI-powered CV analysis.

Phase 3: Feature Expansion (January - February 2026). The primary feature planned for this phase is CV-based recommendation. When users upload their CV or resume, the system will analyze it in conjunction with the applications they have submitted, as detected from their email data. The AI will identify patterns between the user's qualifications and the positions where they have progressed furthest, then provide personalized recommendations on how to enhance their CV for better results. For example, if the system detects that a user consistently receives coding test invitations from fintech companies but rejections from consulting firms, it may suggest emphasizing technical skills and quantitative achievements while recommending adjustments to better appeal to consulting recruiters.

The second major feature is social media sharing functionality. Users will be able to export their application journey visualizations—including the Sankey diagram showing their progress flow and the statistics dashboard—as shareable images or interactive reports. Integration with LinkedIn will allow users to share their job search milestones directly to their professional network, potentially encouraging connections to

provide referrals or advice. This feature recognizes that job searching is often a social process, and celebrating progress can maintain motivation during what is frequently a lengthy endeavor.

Additional features under consideration include job recommendation, where the system analyzes the user's successful application patterns and suggests similar positions they may not have considered, and enhanced analytics providing deeper insights into application timing, response patterns by industry, and comparative benchmarks against anonymized aggregate data.

Phase 4: Deployment & Documentation (March - April 2026). The final phase will focus on production deployment and project completion. The system will be migrated to Amazon Web Services (AWS) for reliable cloud hosting, replacing the current local server architecture with a scalable cloud infrastructure. This deployment will enable the application to serve multiple users simultaneously while maintaining the privacy guarantees established in the current design. AWS services such as Lambda for serverless processing, S3 for secure document storage (CV uploads), and RDS or DynamoDB for user data will be evaluated based on cost and performance requirements.

March will also involve comprehensive user testing with HKU students actively engaged in job searching, gathering feedback on both the core tracking features and the new CV recommendation system. The final report will document the complete system architecture, methodology, results, and lessons learned. Presentation preparation will follow, culminating in the final project demonstration and submission in April.

Risk mitigation strategies have been identified for the upcoming features. CV analysis requires careful prompt engineering to provide genuinely useful recommendations rather than generic advice, which will be addressed through iterative testing with real user CVs. Social sharing must respect user privacy by defaulting to anonymized or aggregated views unless users explicitly choose to share detailed information. AWS deployment costs will be managed through careful service selection and usage monitoring, with the option to maintain a hybrid architecture where computationally intensive AI processing remains local while authentication and data storage move to the cloud.

6. References

- [1] CareerBuilder, "Job Seeker Behavior Study," CareerBuilder Research, 2023.
- [2] Google, "Gmail API Documentation," Google Developers, 2024. Available: <https://developers.google.com/gmail/api>
- [3] OpenAI, "GPT-4o-mini Model Documentation," OpenAI Platform, 2024. Available: <https://platform.openai.com/docs>
- [4] React, "React Documentation," React, 2024. Available: <https://react.dev>
- [5] Plotly, "Plotly.js Documentation," Plotly, 2024. Available: <https://plotly.com/javascript>
- [6] Tailwind Labs, "Tailwind CSS Documentation," Tailwind CSS, 2024. Available: <https://tailwindcss.com/docs>
- [7] Pallets, "Flask Documentation," Flask, 2024. Available: <https://flask.palletsprojects.com>